读享 V1.0 软件

前 30 页

著作权人:XX 有限公司

生成日期:2026-06-15

摘选总行数:1500

文件清单:

- person/notfresh/readingshare/MainActivity.java
- person/notfresh/readingshare/ShareUtil.java
- person/notfresh/readingshare/ui/home/HomeFragment.java

```
1  package person.notfresh.readingshare;
2
3  import android.annotation.SuppressLint;
4  import android.os.Bundle;
5  import android.view.LayoutInflater;
6  import android.view.View;
7  import android.view.Menu;
8  import android.view.MenuItem;
9  import android.content.Intent;
10 import android.net.Uri;
11
12 import java.util.ArrayList;
13 import java.util.List;
14 import java.util.Map;
15
16 import android.util.Log;
17 import android.content.pm.PackageManager;
18 import android.content.pm.ResolveInfo;
19
20 import com.google.android.flexbox.FlexboxLayout;
21 import com.google.android.material.snackbar.Snackbar;
22 import com.google.android.material.navigation.NavigationView;
23
24 import androidx.navigation.NavController;
25 import androidx.navigation.NavDestination;
26 import androidx.navigation.Navigation;
27 import androidx.navigation.ui.AppBarConfiguration;
28 import androidx.navigation.ui.NavigationUI;
29 import androidx.drawerlayout.widget.DrawerLayout;
30 import androidx.appcompat.app.AppCompatActivity;
31 import androidx.fragment.app.Fragment;
32
33 import person.notfresh.readingshare.databinding.ActivityMainBinding;
34 import person.notfresh.readingshare.db.LinkDao;
35 import person.notfresh.readingshare.model.LinkItem;
36 import person.notfresh.readingshare.ui.home.HomeFragment;
37 import android.content.ClipboardManager;
38 import android.content.ClipData;
39 import android.app.AlertDialog;
40 import android.widget.EditText;
41 import android.text.TextUtils;
42 import java.io.BufferedReader;
43 import java.io.InputStreamReader;
44 import java.net.URL;
45 import java.nio.charset.StandardCharsets;
46 import android.os.Handler;
47 import android.os.Looper;
48 import java.util.concurrent.ExecutorService;
49 import java.util.concurrent.Executors;
50 import java.util.regex.Matcher;
```

```
import java.util.regex.Pattern;
import android.content.Context;
import android.Manifest;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import android.app.ActivityManager;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import java.net.URLConnection;
import android.content.SharedPreferences;
import androidx.core.view.GravityCompat;
import person.notfresh.readingshare.util.BilibiliUrlConverter;
import person.notfresh.readingshare.util.CrawlUtil;
import person.notfresh.readingshare.util.RecentTagsManager;
import person.notfresh.readingshare.util.StringUtil;

import java.io.IOException;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;
import android.widget.Spinner;
import android.widget.ArrayAdapter;
import java.io.File;
import person.notfresh.readingshare.util.SubjectUtil;
import person.notfresh.readingshare.db.SubjectDao;
import person.notfresh.readingshare.core.model.Subject;

public class MainActivity extends AppCompatActivity {

    private AppBarConfiguration mAppBarConfiguration;
    private ActivityMainBinding binding;
    private LinkDao linkDao;
    private boolean hasFocus = false;
    private String lastClipboardText = "";  // ■■■■■■■■■■■■■■■■■■■■■
    private NavController navController;  // ■ navController ■■■■■■■■

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        try {
            binding = ActivityMainBinding.inflate(getLayoutInflater());
            setContentView(binding.getRoot());

            linkDao = new LinkDao(this);
            linkDao.open();

            setSupportActionBar(binding.appBarMain.toolbar);

            DrawerLayout drawer = binding.drawerLayout;
```

```
101             NavigationView navigationView = binding.navView;
102
103             // ■■■■■■■■
104             navController = Navigation.findNavController(this, R.id.nav_host_fra
105
106             // ■■■■■■■■■DocumentViewerActivity■■■■■■■
107             handleNavigation(getIntent());
108
109             // ■■■■■ AppBarConfiguration
110             // ■■■nav_tags■■■■■■■■■■■nav_home
111             mAppBarConfiguration = new AppBarConfiguration.Builder(
112                     R.id.nav_home, R.id.nav_slideshow, R.id.nav_rss, R.id.nav_ar
113                     .setOpenableLayout(drawer)
114                     .build();
115
116             NavigationUI.setupActionBarWithNavController(this, navController, mA
117             NavigationUI.setupWithNavController(navigationView, navController);
118
119             // ■■■■■■■■■■■
120             View headerView = navigationView.getHeaderView(0);
121             headerView.setOnClickListener(v -> {
122                 drawer.closeDrawer(GravityCompat.START);
123                 new Handler().postDelayed(() -> {
124                     Intent intent = new Intent(this, UserProfileActivity.class);
125                     startActivity(intent);
126                 }, 250);
127             });
128
129             // ■■■■■■■
130             handleIntent(getIntent());
131
132             // ■■■■■■■■■■
133             navigationView.setNavigationItemSelectedListener(item -> {
134                 int id = item.getItemId();
135                 drawer.closeDrawer(GravityCompat.START);
136
137                 new Handler().postDelayed(() -> {
138                     try {
139                         if (id == R.id.nav_home) {
140                             navController.navigate(R.id.nav_home);
141                         } else if (id == R.id.nav_rss) {
142                             navController.navigate(R.id.nav_rss);
143                         } else if (id == R.id.nav_slideshow) {
144                             navController.navigate(R.id.nav_slideshow);
145                         } else if (id == R.id.nav_archive) {
146                             navController.navigate(R.id.nav_archive);
147                         } else if (id == R.id.nav_documents) {
148                             navController.navigate(R.id.nav_documents);
149                         } else if (id == R.id.nav_subject) {
150                             navController.navigate(R.id.nav_subject);
```

```
151                         }
152                     } catch (Exception e) {
153                         Log.e("MainActivity", "Navigation failed", e);
154                         Toast.makeText(this, "■■■■", Toast.LENGTH_SHORT).show();
155                     }
156                 }, 250);
157 
158                 return true;
159             });
160 
161             checkClipboardPermission();
162 
163             // ■■FAB■■■■
164             binding.appBarMain.fab.setOnClickListener(view -> {
165                 try {
166                     // ■■■■■■■■■■“■■■”■■■■■■■■“■■■”■■
167                     Intent qqMailIntent = new Intent(Intent.ACTION_SENDTO);
168                     qqMailIntent.setPackage("com.tencent.androidqqmail");
169                     qqMailIntent.setClassName(
170                             "com.tencent.androidqqmail",
171                             "com.tencent.qqmail.launcher.third.LaunchComposeMail
172                     );
173                     qqMailIntent.setData(Uri.parse("mailto:notfresh@foxmail.com"
174                     qqMailIntent.putExtra(Intent.EXTRA_SUBJECT, "■■■■");
175                     qqMailIntent.putExtra(Intent.EXTRA_TEXT, "■■■■■■■■■■■■...");
176 
177                     try {
178                         startActivity(qqMailIntent);
179                     } catch (Exception e) {
180                         // ■■■■■■■■■■
181                         Intent emailIntent = new Intent(Intent.ACTION_SENDTO);
182                         emailIntent.setData(Uri.parse("mailto:notfresh@foxmail.c
183                         emailIntent.putExtra(Intent.EXTRA_SUBJECT, "■■■■");
184                         emailIntent.putExtra(Intent.EXTRA_TEXT, "■■■■■■■■■■■■...
185                         startActivity(emailIntent);
186                     }
187                 } catch (Exception e) {
188                     Log.e("MainActivity", "Error opening email app", e);
189                     Toast.makeText(this, "■■■■■■■■", Toast.LENGTH_SHORT).show();
190                 }
191             });
192 
193         } catch (Exception e) {
194             Log.e("MainActivity", "onCreate failed", e);
195             Toast.makeText(this, "■■■■■■: " + e.getMessage(), Toast.LENGTH_LONG)
196             finish();
197         }
198     }
199 
200     @Override
```

```
201      public void onWindowFocusChanged(boolean hasFocus) { //@mark
202          super.onWindowFocusChanged(hasFocus);
203          this.hasFocus = hasFocus;
204
205          if (hasFocus) {
206              // ■■■■■■■■■■■■■■■■
207              checkClipboard(); //@Def Line300
208          }
209      }
210
211      @Override
212      public boolean onCreateOptionsMenu(Menu menu) {
213          // ■ Fragment ■■■■
214          return super.onCreateOptionsMenu(menu);
215      }
216
217      @Override
218      protected void onResume() {
219          super.onResume();
220          updateNavHeader();
221      }
222
223      @Override
224      protected void onDestroy() {
225          super.onDestroy();
226          if (linkDao != null) {
227              linkDao.close();
228          }
229      }
230
231      @Override
232      public boolean onOptionsItemSelected(MenuItem item) {
233          // ■■ Fragment ■■■■■■■
234          Fragment currentFragment = getSupportFragmentManager().findFragmentById(
235          if (currentFragment != null && currentFragment.onOptionsItemSelected(ite
236              return true;
237          }
238          return super.onOptionsItemSelected(item);
239      }
240
241      @Override
242      public boolean onSupportNavigateUp() {
243          return NavigationUI.navigateUp(navController, mAppBarConfiguration)
244                  || super.onSupportNavigateUp();
245      }
246
247
248      @Override
249      protected void onNewIntent(Intent intent) {
250          super.onNewIntent(intent);
```

```
251         setIntent(intent);
252         // ■■■■■■■DocumentViewerActivity■■■■■
253         if (navController != null) {
254             handleNavigation(intent);
255         }
256         handleIntent(intent);
257     }
258 
259     /**
260      * ■■■■■■■
261      */
262     private void handleNavigation(Intent intent) {
263         if (intent == null || navController == null) {
264             return;
265         }
266 
267         String navigateTo = intent.getStringExtra("navigate_to");
268         int destinationId;
269 
270         if ("documents".equals(navigateTo)) {
271             // ■■■■■■PDF■■■■■■■■■■■■■
272             destinationId = R.id.nav_documents;
273         } else if ("home".equals(navigateTo)) {
274             // ■■■■■■WebView■■■■■■■■■■
275             destinationId = R.id.nav_home;
276         } else {
277             // ■■■■■Tab■■■■■■
278             SharedPreferences prefs = getPreferences(Context.MODE_PRIVATE);
279             int defaultTab = prefs.getInt("default_tab", 0); // ■■■0■■■■■
280 
281             // ■■■■■■■■■■■ID
282             switch (defaultTab) {
283                 case 0: // ■■
284                     destinationId = R.id.nav_home;
285                     break;
286                 case 2: // ■■
287                     destinationId = R.id.nav_subject;
288                     break;
289                 case 3: // RSS
290                     destinationId = R.id.nav_rss;
291                     break;
292                 case 4: { // ■■■■■/■■/RSS
293                     int r = (int) (Math.random() * 3);
294                     destinationId = (r == 0)
295                         ? R.id.nav_home
296                         : (r == 1 ? R.id.nav_subject : R.id.nav_rss);
297                     break;
298                 }
299                 default: // ■■■■■■■■■■■case 1■■■■■
300                     destinationId = R.id.nav_home;
```

```
301                     break;
302             }
303         }
304 
305         // ■■■■■■■
306         try {
307             navController.navigate(destinationId);
308         } catch (Exception e) {
309             Log.e("MainActivity", "■■■■", e);
310         }
311     }
312 
313     private void handleIntent(Intent intent) {
314         String action = intent.getAction();
315         String type = intent.getType();
316 
317         if (Intent.ACTION_SEND.equals(action) && type != null) {
318             if ("text/plain".equals(type)) {
319                 String sharedText = intent.getStringExtra(Intent.EXTRA_TEXT);
320                 String title = intent.getStringExtra(Intent.EXTRA_SUBJECT);
321 
322                 // ■■Intent■■■■■■■
323                 Log.d("ShareIntent", "Action: " + action);
324                 Log.d("ShareIntent", "Type: " + type);
325                 Log.d("ShareIntent", "Text: " + sharedText);
326                 Log.d("ShareIntent", "Subject: " + title);
327                 if (intent.getComponent() != null) {
328                     Log.d("ShareIntent", "Package: " + intent.getComponent().get
329                 }
330                 Log.d("ShareIntent", "Data: " + intent.getDataString());
331                 Bundle extras = intent.getExtras();
332                 if (extras != null) {
333                     for (String key : extras.keySet()) {
334                         Log.d("ShareIntent", "Extra: " + key + " = " + extras.ge
335                     }
336                 }
337 
338                 // ■■■■■■Intent■■
339                 Log.d("ShareIntent", "=== ■■■■■■ ===");
340                 Log.d("ShareIntent", "Intent: " + intent.toString());
341                 Log.d("ShareIntent", "Categories: " + intent.getCategories());
342                 Log.d("ShareIntent", "Flags: " + Integer.toHexString(intent.getF
343                 Log.d("ShareIntent", "Scheme: " + intent.getScheme());
344 
345                 if (intent.getComponent() != null) {
346                     Log.d("ShareIntent", "Component: " + intent.getComponent().f
347                 }
348 
349                 // ■■■■■■■■■■Intent■■■
350                 PackageManager pm = getPackageManager();
```

```
351                 List<ResolveInfo> activities = pm.queryIntentActivities(intent,
352                 for (ResolveInfo info : activities) {
353                     Log.d("ShareIntent", "■■■■■■■: " + info.activityInfo.packageN
354                     Log.d("ShareIntent", "Activity: " + info.activityInfo.name);
355                 }
356 
357                 if (sharedText != null) {
358                     // ■■■■■■■■■■■■
359                     String sourceApp = "";
360                     if (intent.getComponent() != null) {
361                         sourceApp = intent.getComponent().getPackageName();
362                     }
363 
364                     // ■■■■■
365                     String initialTitle = (title == null || title.isEmpty())
366                             ? StringUtil.extractTitle(sharedText) : title;
367 
368                     // ■■■■■ Activity ■■■■■■■■■■
369                     StringBuilder targetActivityInfo = new StringBuilder();
370                     for (ResolveInfo info : activities) {
371                         targetActivityInfo.append(info.activityInfo.packageName)
372                                 .append("/")
373                                 .append(info.activityInfo.name)
374                                 .append(";");
375                     }
376 
377                     // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
378                     showSaveLinkDialog(
379                             sharedText,
380                             initialTitle,
381                             sourceApp,
382                             intent.toString(),
383                             targetActivityInfo.toString(),
384                             false
385                     );
386                 }
387             }
388         }
389     }
390 
391     private void checkClipboardPermission() {
392         // ■■■■■■■■■■■■■■■■
393         checkClipboard();
394     }
395 
396     private void checkClipboard() {
397         try {
398             // ■■■■■■■■■■■■■■■
399             if (!hasFocus || !isAppInForeground()) {
400                 Log.d("Clipboard", "App is not in focus or foreground");
```

```
                return;
            }

            // ■■■■■■■■■■■■■■■■■■
            new Handler(Looper.getMainLooper()).postDelayed(() -> {
                try {
                    Log.d("Clipboard", "Checking clipboard...");
                    ClipboardManager clipboard = (ClipboardManager) getSystemSer
                    if (clipboard == null || !clipboard.hasPrimaryClip()) {
                        return;
                    }
                    ClipData clipData = clipboard.getPrimaryClip();
                    if (clipData == null || clipData.getItemCount() == 0) {
                        return;
                    }
                    CharSequence clipTextSequence = clipData.getItemAt(0).getTex
                    if (clipTextSequence == null) {
                        return;
                    }
                    SharedPreferences prefs = getSharedPreferences("clipboard_pr
                    String savedText = prefs.getString("last_clipboard_text", ""
                    if (!clipTextSequence.toString().equals(savedText)) {
                        lastClipboardText = clipTextSequence.toString();
                        handleClipboardText(lastClipboardText); //@Def Line376
                    }
                    prefs.edit().putString("last_clipboard_text", clipTextSequen

                } catch (Exception e) {
                    Log.e("Clipboard", "Error checking clipboard", e);
                    e.printStackTrace();
                }
            }, 500);

        } catch (Exception e) {
            Log.e("Clipboard", "Error in checkClipboard", e);
            e.printStackTrace();
        }
    }

    private boolean isAppInForeground() {
        ActivityManager activityManager = (ActivityManager) getSystemService(Con
        List<ActivityManager.RunningAppProcessInfo> appProcesses = activityManag
        if (appProcesses == null) return false;

        String packageName = getPackageName();
        for (ActivityManager.RunningAppProcessInfo appProcess : appProcesses) {
            if (appProcess.importance == ActivityManager.RunningAppProcessInfo.I
                    && appProcess.processName.equals(packageName)) {
                return true;
            }
```

```
451          }
452          return false;
453      }
454
455      private void handleClipboardText(String text) {
456          if (TextUtils.isEmpty(text)) {
457              return;
458          }
459
460          Log.d("LinkParser", "Processing text: " + text);
461
462          // ■■■■■ http(s) URL
463          Pattern urlPattern = Pattern.compile("https?://[^\\s,■]+");
464          Matcher urlMatcher = urlPattern.matcher(text);
465
466          if (urlMatcher.find()) {
467              // ■■URL
468              String url = urlMatcher.group(0);
469              // ■■URL■■■■■■■■
470              String cleanUrl = url.split("\\?")[0];
471              // ■■■■■.xml■■■■■■■■■
472              if (cleanUrl.toLowerCase().endsWith(".xml")) {
473                  return;
474              }
475
476              // ■■URL■■■■■■■■■
477              String title = text.substring(0, urlMatcher.start()).trim();
478
479              // ■■■■■■■■■■■■■■■
480              if (TextUtils.isEmpty(title)) {
481                  title = "■■■";
482              } else if (title.length() > 20) {
483                  title = title.substring(0, 20) + "...";
484              }
485
486              Log.d("LinkParser", "Extracted title: " + title);
487              Log.d("LinkParser", "Extracted URL: " + url);
488
489              // ■■■■■■■
490              final String finalTitle = title;
491              final String finalUrl = url;
492              new Handler(Looper.getMainLooper()).post(() -> {
493                  showSaveLinkDialog(finalUrl, finalTitle);
494              });
495          }
496      }
497
498      /** ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
499      private void showSaveLinkDialog(String url, String initialTitle) {
500          showSaveLinkDialog(url, initialTitle, "clipboard", "clipboard", "", true
```

```
501      }
502
503      /**
504       * ■■■■■■■■■■■■■■■■■■■■■■■■■
505       * @param clearClipboardOnSave ■■■■■■■■ true■■■■■■■■■■■■■■■■ false
506       */
507      private void showSaveLinkDialog(String url, String initialTitle,
508              String sourceApp, String originalIntent, String targetActivity,
509              boolean clearClipboardOnSave) {
510          View dialogView = getLayoutInflater().inflate(R.layout.dialog_save_link,
511          EditText titleInput = dialogView.findViewById(R.id.edit_title);
512          EditText urlInput = dialogView.findViewById(R.id.edit_url);
513          EditText tagsInput = dialogView.findViewById(R.id.edit_tags);
514          FlexboxLayout recentTagsContainer = dialogView.findViewById(R.id.recent_
515          Spinner subjectSpinner = dialogView.findViewById(R.id.spinner_save_to_su
516
517          // ■■■■■■■■■
518          titleInput.setHint(initialTitle);  // ■■ hint ■■■■■■■■■
519          urlInput.setText(url);
520
521          // ■■■■■■■■■■■
522          titleInput.requestFocus();
523
524          // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
525          final List<Subject> subjectList = new ArrayList<>();
526          List<String> subjectLabels = new ArrayList<>();
527          subjectLabels.add("■■■■■■■");
528          SubjectDao subjectDao = new SubjectDao(this);
529          subjectDao.open();
530          try {
531              List<Subject> all = subjectDao.getAllSubjects();
532              for (Subject s : all) {
533                  subjectList.add(s);
534                  subjectLabels.add(s.getTitle());
535              }
536          } finally {
537              subjectDao.close();
538          }
539          ArrayAdapter<String> subjectAdapter = new ArrayAdapter<>(this,
540                  android.R.layout.simple_spinner_item, subjectLabels);
541          subjectAdapter.setDropDownViewResource(android.R.layout.simple_spinner_d
542          subjectSpinner.setAdapter(subjectAdapter);
543
544          // ■■■■■■■■■■
545          Context context = this;
546          List<String> recentTags = RecentTagsManager.getRecentTags(context);
547          if (!recentTags.isEmpty()) {
548              TextView recentTagsLabel = dialogView.findViewById(R.id.text_recent_
549              recentTagsLabel.setVisibility(View.VISIBLE);
550
```

```
551             for (String tag : recentTags) {
552                 TextView tagView = (TextView) LayoutInflater.from(context)
553                         .inflate(R.layout.item_recent_tag, recentTagsContainer,
554                 tagView.setText(tag);
555                 tagView.setOnClickListener(v -> {
556                     String currentText = tagsInput.getText().toString().trim();
557                     // ■■■■■■■■■■■■■■■■■■■■■■■■
558                     if (!currentText.isEmpty() && !currentText.endsWith(",") &&
559                         tagsInput.setText(currentText + "■" + tag);
560                     } else {
561                         // ■■■■■■■■■■■■■■■■■■
562                         tagsInput.setText(currentText + tag);
563                     }
564                     // ■■■■■■■■
565                     tagsInput.setSelection(tagsInput.getText().length());
566                 });
567                 recentTagsContainer.addView(tagView);
568             }
569         }
570 
571         AlertDialog dialog = new AlertDialog.Builder(this)
572                 .setTitle("■■■■")
573                 .setView(dialogView)
574                 .setPositiveButton("■■", (d, which) -> {
575                     // ■■■■■■■■■■■■ hint ■■■■■■
576                     String title = titleInput.getText().toString();
577                     if (TextUtils.isEmpty(title)) {
578                         title = initialTitle;
579                     }
580                     String finalUrl = urlInput.getText().toString();
581 
582                     if (!TextUtils.isEmpty(title) && !TextUtils.isEmpty(finalUrl
583                         LinkItem newLink = new LinkItem(
584                             title,
585                             finalUrl,
586                             sourceApp,
587                             originalIntent,
588                             targetActivity
589                         );
590                         String[] tags = tagsInput.getText().toString().split("[,
591                         List<String> tagList = new ArrayList<>();
592                         for (String tag : tags) {
593                             String trimmedTag = tag.trim();
594                             if (!trimmedTag.isEmpty()) {
595                                 tagList.add(trimmedTag);
596                             }
597                         }
598                         newLink.setTags(tagList);
599                         linkDao.insertLink(newLink);
600 
```

```
601                         // ■■■■■■■■■■■■■■■■■■■■■■
602                         int subjectPos = subjectSpinner.getSelectedItemPosition(
603                         if (subjectPos > 0 && subjectPos <= subjectList.size())
604                             Subject chosen = subjectList.get(subjectPos - 1);
605                             SubjectUtil.addLinkToSubjectById(this, chosen.getId(
606                         }
607
608                         // ■■■■■■■■■■■■■■■■■■■■■■■■
609                         if (clearClipboardOnSave) {
610                             ClipboardManager clipboard = (ClipboardManager) getS
611                             clipboard.setPrimaryClip(ClipData.newPlainText("", "
612                         }
613
614                         Snackbar.make(binding.getRoot(), "■■■■" + title,
615                                 Snackbar.LENGTH_LONG).show();
616
617                         NavController navController = Navigation.findNavControll
618                                 R.id.nav_host_fragment_content_main);
619                         navController.navigate(R.id.nav_home);
620
621                         // ■■ HomeFragment ■■■■■■■■■■■■■■■■■■■■
622                         navController.addOnDestinationChangedListener(new NavCon
623                             @Override
624                             public void onDestinationChanged(@NonNull NavControl
625                                                              @NonNull NavDestinat
626                                                              @Nullable Bundle arg
627                                 if (destination.getId() == R.id.nav_home) {
628                                     // ■■ HomeFragment ■■■■■■■■■■
629                                     Fragment fragment = getSupportFragmentManage
630                                             .findFragmentById(R.id.nav_host_frag
631                                     if (fragment instanceof HomeFragment) {
632                                         ((HomeFragment) fragment).recordLinkAddT
633                                     }
634                                     navController.removeOnDestinationChangedList
635                                 }
636                             }
637                         });
638                     }
639                 })
640                 .setNegativeButton("■■", (dialog1, which) -> {
641                     // ■■■■■■■■■■■■■■■■■■■■
642                     // ClipboardManager clipboard = (ClipboardManager) getSystem
643                     // clipboard.setPrimaryClip(ClipData.newPlainText("", ""));
644                 })
645                 .create();
646
647         // ■■■■■■■■■■
648         dialog.setOnDismissListener(dialogInterface -> {
649             // ■■■■■■■■■■■■■■■■■■■■■■■■
650             // ClipboardManager clipboard = (ClipboardManager) getSystemService(
```

```
651             // clipboard.setPrimaryClip(ClipData.newPlainText("", ""));
652         });
653 
654         dialog.show();
655 
656         // ■■■■■■■■■■
657         fetchTitleFromUrl(url, titleInput);
658     }
659 
660     private void fetchTitleFromUrl(String url, EditText titleInput) {
661         ExecutorService executor = Executors.newSingleThreadExecutor();
662         Handler handler = new Handler(Looper.getMainLooper());
663 
664         executor.execute(() -> {
665 
666             String title = "";
667             try {
668                 if (url.contains("weixin.qq.com")) {
669                     title = CrawlUtil.getWeixinArticleTitle(url);
670                 } else {
671                     title = CrawlUtil.fetchTitleCommon(url);
672                 }
673             } catch (Exception e) {
674                 Log.e("FetchTitle", "Error fetching title", e);
675             }
676 
677             final String finalTitle = title;
678             handler.post(() -> {
679                 if (!TextUtils.isEmpty(finalTitle)) {
680                     titleInput.setText(finalTitle);
681                 }
682             });
683         });
684     }
685 
686     @SuppressLint("ResourceType")
687     private void updateNavHeader() {
688         try {
689             NavigationView navigationView = binding.navView;
690             View headerView = navigationView.getHeaderView(0);
691             ImageView profileImage = headerView.findViewById(R.id.nav_header_ima
692             TextView usernameText = headerView.findViewById(R.id.nav_header_user
693             TextView emailText = headerView.findViewById(R.id.nav_header_email);
694 
695             SharedPreferences prefs = getSharedPreferences("UserProfile", MODE_P
696             String username = prefs.getString("username", getString(R.string.nav
697             String email = prefs.getString("email", getString(R.string.nav_heade
698             String imageUri = prefs.getString("profile_image", "");
699 
700             usernameText.setText(username);
```

```
701             emailText.setText(email);
702
703             // ■■■■■■■■■■
704             if (!TextUtils.isEmpty(imageUri)) {
705                 try {
706                     Uri uri = Uri.parse(imageUri);
707                     if (uri.getPath() != null && new File(uri.getPath()).exists(
708                         profileImage.setImageURI(null); // ■■■■■■■■
709                         profileImage.setImageURI(uri);
710                     } else {
711                         // ■■■■■■■■■■■■■■■■URI■■■■■■logo
712                         prefs.edit().remove("profile_image").apply();
713                         profileImage.setImageResource(R.mipmap.ic_launcher);
714                     }
715                 } catch (Exception e) {
716                     Log.e("MainActivity", "Error loading profile image", e);
717                     profileImage.setImageResource(R.mipmap.ic_launcher);
718                 }
719             }
720             // ■■■■■■■■■■■■■■■■■■■■■■ic_launcher■■XML■■■■■■
721         } catch (Exception e) {
722             Log.e("MainActivity", "Error updating nav header", e);
723         }
724     }
725
726 }
```

文件:person/notfresh/readingshare/ShareUtil.java 起始行:1

```
1  // MISSING: person/notfresh/readingshare/ShareUtil.java
```

```
  1  package person.notfresh.readingshare.ui.home;
  2
  3  import android.app.Activity;
  4  import android.content.Intent;
  5  import android.graphics.Bitmap;
  6  import android.net.Uri;
  7  import android.os.Bundle;
  8  import android.text.Editable;
  9  import android.text.TextWatcher;
 10  import android.util.Log;
 11  import android.view.LayoutInflater;
 12  import android.view.Menu;
 13  import android.view.MenuInflater;
 14  import android.view.MenuItem;
 15  import android.view.View;
 16  import android.view.ViewGroup;
 17  import android.widget.EditText;
 18  import android.widget.PopupWindow;
 19  import android.widget.Toast;
 20
 21  import androidx.annotation.NonNull;
 22  import androidx.annotation.Nullable;
 23  import androidx.fragment.app.Fragment;
 24  import androidx.navigation.Navigation;
 25  import androidx.recyclerview.widget.ItemTouchHelper;
 26  import androidx.recyclerview.widget.LinearLayoutManager;
 27  import androidx.recyclerview.widget.RecyclerView;
 28
 29  import com.google.android.flexbox.FlexboxLayoutManager;
 30  import com.google.android.flexbox.FlexDirection;
 31  import com.google.android.flexbox.FlexWrap;
 32
 33  import com.google.android.material.snackbar.Snackbar;
 34
 35  import person.notfresh.readingshare.R;
 36  import person.notfresh.readingshare.adapter.LinksAdapter;
 37  import person.notfresh.readingshare.adapter.SearchHistoryAdapter;
 38  import person.notfresh.readingshare.adapter.TagsAdapter;
 39  import person.notfresh.readingshare.databinding.FragmentHomeBinding;
 40  import person.notfresh.readingshare.db.LinkDao;
 41  import person.notfresh.readingshare.db.SearchHistoryManager;
 42  import person.notfresh.readingshare.db.SubjectDao;
 43  import person.notfresh.readingshare.model.LinkItem;
 44  import person.notfresh.readingshare.model.SearchHistoryItem;
 45  import person.notfresh.readingshare.core.model.SubjectItem;
 46  import person.notfresh.readingshare.core.model.SubjectUtil;
 47  import person.notfresh.readingshare.ui.subject.SelectSubjectDialog;
 48  import person.notfresh.readingshare.ClickStatisticsActivity;
 49  import person.notfresh.readingshare.util.ShareUtil;
 50  import person.notfresh.readingshare.util.ImageUtil;
```

```
 51
 52  import android.content.Context;
 53  import android.content.SharedPreferences;
 54  import android.text.TextUtils;
 55  import android.widget.ImageButton;
 56  import android.widget.ImageView;
 57  import android.widget.ScrollView;
 58  import android.widget.TextView;
 59  import androidx.appcompat.app.AlertDialog;
 60
 61  import org.slf4j.Logger;
 62  import org.slf4j.LoggerFactory;
 63
 64  import java.util.ArrayList;
 65  import java.util.Collections;
 66  import java.util.Date;
 67  import java.util.HashSet;
 68  import java.util.List;
 69  import java.util.Locale;
 70  import java.util.Map;
 71  import java.util.Set;
 72  import java.util.TreeMap;
 73  import java.text.SimpleDateFormat;
 74  import java.util.concurrent.atomic.AtomicInteger;
 75
 76  /**
 77   * ■■Fragment
 78   *
 79   * ■■■■■
 80   * - ■■■■■■■■■■■■■■■■■■■■
 81   * - ■■■■■■■■■■■■■TagsFragment■■■
 82   * - ■■■■■■■■■■■■■■■■■■■
 83   * - ■■■■■■
 84   *
 85   * ■■■■■
 86   * - ■■0-2■■TagsFragment■■■■■■HomeFragment
 87   * - ■■3■■■■■■
 88   * - ■■4■■■■■■■■■■■■
 89   *
 90   * @see TagsFragment■■■■■■■■■■■■Fragment■
 91   */
 92  public class HomeFragment extends Fragment implements LinksAdapter.OnLinkActionL
 93
 94      private static final String TAG = "HomeFragment";  // Logcat■■■■■
 95      private static final int REQUEST_CODE_PICK_ICON = 1001;
 96      private static final Logger log = LoggerFactory.getLogger(HomeFragment.class
 97
 98      private FragmentHomeBinding binding;
 99      private LinksAdapter adapter;
100      private LinkDao linkDao;
```

```
101      private boolean isSelectionMode = false;
102      private MenuItem shareMenuItem;
103      private MenuItem closeSelectionMenuItem;
104      private MenuItem enterSelectionMenuItem;
105      private MenuItem selectAllMenuItem;  // ████████TagsFragment███
106      private MenuItem addTagMenuItem;  // ██████████TagsFragment███
107      private MenuItem sortMenuItem;  // ████████TagsFragment███
108      private MenuItem exitSortMenuItem;  // ██████████TagsFragment███
109      private EditText searchEditText;
110
111      // ========== ██████ ==========
112      private static final long SEARCH_DEBOUNCE_MS = 300L;
113      private static final long SEARCH_BLUR_HIDE_DELAY_MS = 200L;
114      private static final int SEARCH_HISTORY_POPUP_MAX_DP = 300;
115      private static final int SEARCH_DEFAULT_END_PADDING_DP = 8;
116      private static final int SEARCH_CLEAR_END_PADDING_DP = 36;
117
118      private SearchHistoryManager searchHistoryManager;
119      private List<SearchHistoryItem> historyCache = new ArrayList<>();
120      private PopupWindow historyPopup;
121      private SearchHistoryAdapter historyAdapter;
122      private final Runnable debounceSearchRunnable = new Runnable() {
123          @Override public void run() { performSearch(searchEditText.getText().toS
124      };
125      private final Runnable hidePopupRunnable = new Runnable() {
126          @Override public void run() { dismissHistoryPopup(); }
127      };
128      private ImageButton searchClearButton;
129
130      // ████████████████
131      private LinkItem pendingIconItem;
132      private String pendingIconUrl;
133
134      // ========== ██████████████TagsFragment███==========
135      private static final String PREF_NAME = "TagsPreferences";
136      private static final String KEY_SELECTED_TAGS = "selectedTags";
137      private static final String KEY_NO_TAG_SELECTED = "noTagSelected";
138      private static final String NO_TAG = "NO_TAG";  // ████"███"██
139      private static final String PREF_HIGHLIGHTED_TAGS = "highlighted_tags";
140      private static final int FIXED_TAGS_COUNT = 10;  // ██████████
141      private static final int COLLAPSED_HEIGHT_DP = 120;  // ████
142      private static final int SORT_MODE_MAX_HEIGHT_DP = 400;  // ████████████dp█
143
144      // █████████
145      private static final String KEY_SHUFFLE_MODE = "shuffle_mode";
146      private static final long LINK_ADD_INTERVAL_MS = 5000;  // 5████████████
147
148      private RecyclerView tagsRecyclerView;
149      private RecyclerView tagsRecyclerViewCollapsed;  // ██████
150      private TagsAdapter tagsAdapter;
```

```
151      private TagsAdapter tagsAdapterCollapsed;  // ■■■■■■■■■■
152      private ItemTouchHelper itemTouchHelper;
153      private View expandMoreTagsButton;  // ■■■■■■■■■
154      private TextView textExpandMore;
155      private ImageView iconExpandMore;
156      private boolean isMoreTagsExpanded = false;  // ■■■■■■■■■
157      private Set<String> selectedTagNames = new HashSet<>();  // ■■Set■■■■■■■■■■
158      private boolean isSortMode = false;  // ■■■■■■■
159      private ScrollView tagsScrollView;
160      private View toggleTagsButton;
161      private ImageView arrowIndicator;
162      private boolean isTagsExpanded = false;  // ■■■■
163      private Set<String> highlightedTags = new HashSet<>();  // ■■■■■■■■
164      private View tagsContainer;  // ■■■■■■■■■■■■/■■■
165      private boolean isShuffleMode = false;  // ■■■■■■■■■
166      private MenuItem shuffleMenuItem;  // ■■■■■■
167      private MenuItem exitShuffleMenuItem;  // ■■■■■■■■
168      private long lastLinkAddTime = 0;  // ■■■■■■■■■■■■■■■■■■■■■■■■■
169
170      @Override
171      public void onCreate(@Nullable Bundle savedInstanceState) {
172          super.onCreate(savedInstanceState);
173          Log.d(TAG, "onCreate: start");
174          try {
175              setHasOptionsMenu(true);
176              Log.d(TAG, "onCreate: setHasOptionsMenu(true) success");
177          } catch (Exception e) {
178              Log.e(TAG, "onCreate: exception", e);
179              throw e;
180          }
181
182          // ■■■■■■■■■
183          restoreShuffleModeState();
184      }
185
186      @Override
187      public void onCreateOptionsMenu(@NonNull Menu menu, @NonNull MenuInflater in
188          Log.d(TAG, "onCreateOptionsMenu: start");
189          try {
190              menu.clear();
191              inflater.inflate(R.menu.home_menu, menu);
192              Log.d(TAG, "onCreateOptionsMenu: menu loaded successfully");
193
194              shareMenuItem = menu.findItem(R.id.action_share);
195              closeSelectionMenuItem = menu.findItem(R.id.action_close_selection);
196              enterSelectionMenuItem = menu.findItem(R.id.action_enter_selection);
197              selectAllMenuItem = menu.findItem(R.id.action_select_all);  // ■■■■■■
198              addTagMenuItem = menu.findItem(R.id.action_add_tag);  // ■■■■■■■■
199              sortMenuItem = menu.findItem(R.id.action_sort_tags);  // ■■■■■■
200              exitSortMenuItem = menu.findItem(R.id.action_exit_sort);  // ■■■■■■■■
```

```
201             shuffleMenuItem = menu.findItem(R.id.action_shuffle);  // ■■■■■
202             exitShuffleMenuItem = menu.findItem(R.id.action_exit_shuffle);  // ■
203             MenuItem statisticsMenuItem = menu.findItem(R.id.action_statistics);
204
205             Log.d(TAG, "onCreateOptionsMenu: menu items found - share=" + (share
206                     ", close=" + (closeSelectionMenuItem != null) +
207                     ", enter=" + (enterSelectionMenuItem != null) +
208                     ", selectAll=" + (selectAllMenuItem != null) +
209                     ", addTag=" + (addTagMenuItem != null) +
210                     ", sort=" + (sortMenuItem != null) +
211                     ", exitSort=" + (exitSortMenuItem != null) +
212                     ", shuffle=" + (shuffleMenuItem != null) +
213                     ", exitShuffle=" + (exitShuffleMenuItem != null));
214
215             // ■■■■■■■■■■■■
216             View actionView = requireActivity().findViewById(statisticsMenuItem.
217             if (actionView != null) {
218                 ViewGroup.MarginLayoutParams params = (ViewGroup.MarginLayoutPar
219                 params.rightMargin = getResources().getDimensionPixelSize(R.dime
220                 actionView.setLayoutParams(params);
221             }
222
223            //■■■■■■■■■■■
224         //     MenuItem clickStatsButton = menu.findItem(R.id.action_statistics_
225         //    clickStatsButton.setOnClickListener(v -> {
226         //        // ■■■■■■■■■■
227         //        Intent intent = new Intent(getActivity(), ClickStatisticsActiv
228         //        startActivity(intent);
229         //    });
230
231             // ■■2■■■■■■■■■■■■■■■■■■■■■■■■
232             updateMenuVisibility();
233
234             super.onCreateOptionsMenu(menu, inflater);
235             Log.d(TAG, "onCreateOptionsMenu: completed");
236         } catch (Exception e) {
237             Log.e(TAG, "onCreateOptionsMenu: exception", e);
238             // ■■■■■■■■■■■■■
239             super.onCreateOptionsMenu(menu, inflater);
240         }
241     }
242
243     @Override
244     public boolean onOptionsItemSelected(@NonNull MenuItem item) {
245         int id = item.getItemId();
246         if (id == R.id.action_close_selection) {
247             toggleSelectionMode();  // ■■■■■■■
248             return true;
249         } else if (id == R.id.action_enter_selection) {
250             toggleSelectionMode();  // ■■■■■■■
```

```
251             return true;
252         } else if (id == R.id.action_share_text) {
253             shareAsText();
254             return true;
255         } else if (id == R.id.action_share_json) {
256             shareAsFile(true);  // true ■■ JSON
257             return true;
258         } else if (id == R.id.action_share_csv) {
259             shareAsFile(false);  // false ■■ CSV
260             return true;
261         } else if (id == R.id.action_add_to_subject) {
262             addToSubject();
263             return true;
264         } else if (id == R.id.action_statistics) {
265             // ■■■■■■■
266             Navigation.findNavController(requireView())
267                     .navigate(R.id.action_nav_home_to_nav_statistics);
268             return true;
269         } else if (id == R.id.action_statistics_read) {
270             // ■■■■■■■■■■
271             Intent intent = new Intent(getActivity(), ClickStatisticsActivity.cl
272             startActivity(intent);
273             return true;
274         } else if (id == R.id.action_add_tag) {
275             // ■■■■■■TagsFragment■■■
276             showAddTagDialog();
277             return true;
278         } else if (id == R.id.action_sort_tags) {
279             // ■■■■■■■■TagsFragment■■■
280             toggleSortMode();
281             return true;
282         } else if (id == R.id.action_exit_sort) {
283             // ■■■■■■■■TagsFragment■■■
284             toggleSortMode();
285             return true;
286         } else if (id == R.id.action_select_all) {
287             // ■■■■TagsFragment■■■
288             selectAllItems();
289             return true;
290         } else if (id == R.id.action_shuffle) {
291             if (isShuffleMode) {
292                 // ■■■■■■■■■■■■
293                 reshuffleLinks();
294             } else {
295                 // ■■■■■■■
296                 toggleShuffleMode();
297             }
298             return true;
299         } else if (id == R.id.action_exit_shuffle) {
300             // ■■■■■■■
```

```
301             toggleShuffleMode();
302             return true;
303         }
304         return super.onOptionsItemSelected(item);
305     }
306 
307     @Override
308     public View onCreateView(@NonNull LayoutInflater inflater,
309                              ViewGroup container, Bundle savedInstanceState) {
310         Log.d(TAG, "onCreateView: start");
311         try {
312             binding = FragmentHomeBinding.inflate(inflater, container, false);
313             View root = binding.getRoot();
314             Log.d(TAG, "onCreateView: layout loaded successfully");
315 
316             // ■■■■■■■■■■
317             String selectedDate = null;
318             if (getArguments() != null) {
319                 selectedDate = getArguments().getString("selected_date");
320             }
321 
322             Log.d(TAG, "onCreateView: initializing LinkDao");
323             linkDao = new LinkDao(requireContext());
324             linkDao.open();
325             Log.d(TAG, "onCreateView: LinkDao opened successfully");
326 
327             Log.d(TAG, "onCreateView: initializing RecyclerView and Adapter");
328             RecyclerView recyclerView = binding.recyclerView;
329             if (recyclerView == null) {
330                 Log.e(TAG, "onCreateView: recyclerView is null!");
331                 throw new IllegalStateException("recyclerView■null");
332             }
333 
334             adapter = new LinksAdapter(requireContext());
335             adapter.setOnLinkActionListener(this);
336             recyclerView.setAdapter(adapter);
337             recyclerView.setLayoutManager(new LinearLayoutManager(requireContext
338             Log.d(TAG, "onCreateView: RecyclerView setup completed");
339 
340             // ■■■■■■■■
341             adapter.enableSwipeActions(recyclerView);
342 
343             // ■■ RecyclerView ■■■■■
344             recyclerView.setOnTouchListener((v, event) -> {
345                 if (searchEditText != null) {
346                     searchEditText.clearFocus();  // ■■■■■■■■
347                 }
348                 return false;
349             });
350 
```

```
351             // ■■■■■
352             Log.d(TAG, "onCreateView: initializing search box");
353             searchEditText = binding.searchEditText;
354             if (searchEditText != null) {
355                 // ■■■■■■■
356                 searchHistoryManager = new SearchHistoryManager(requireContext()
357                 refreshHistoryCache();
358 
359                 searchEditText.addTextChangedListener(new TextWatcher() {
360                     @Override public void beforeTextChanged(CharSequence s, int
361                     @Override public void onTextChanged(CharSequence s, int star
362                     @Override
363                     public void afterTextChanged(Editable s) {
364                         // ■■■■ × ■■■■■■■■■ 300ms ■■■
365                         updateInputChrome();
366                         // ■■■■■■■■ + ■■■
367                         searchEditText.removeCallbacks(debounceSearchRunnable);
368                         searchEditText.postDelayed(debounceSearchRunnable, SEARC
369                     }
370                 });
371 
372                 // focus: ■■■ updateInputChrome ■■■■■■■■■
373                 searchEditText.setOnFocusChangeListener((v, hasFocus) -> {
374                     if (hasFocus) {
375                         updateInputChrome();
376                     } else {
377                         // 200ms ■■■■■■■■■■■■
378                         searchEditText.removeCallbacks(hidePopupRunnable);
379                         searchEditText.postDelayed(hidePopupRunnable, SEARCH_BLU
380                     }
381                 });
382 
383                 // × ■■■■■■■■■ + ■■■■ + ■ updateInputChrome ■■■■■■■
384                 searchClearButton = binding.searchClearButton;
385                 if (searchClearButton != null) {
386                     searchClearButton.setOnClickListener(v -> {
387                         searchEditText.removeCallbacks(debounceSearchRunnable);
388                         searchEditText.setText("");
389                         // ■■■■■setText ■■■ TextWatcher
390                         updateInputChrome();
391                     });
392                 }
393             } else {
394                 Log.w(TAG, "onCreateView: searchEditText is null");
395             }
396 
397             // ========== ■■■■■■■UI■■■2■■■==========
398             Log.d(TAG, "onCreateView: initializing tag management UI");
399             try {
400                 initTagRecyclerView(root);
```

```
401                 Log.d(TAG, "onCreateView: tag management UI initialized successf
402             } catch (Exception e) {
403                 Log.e(TAG, "onCreateView: tag management UI initialization faile
404                 // ■■■■■■■■■■■■■
405             }
406 
407             // ========== ■■■■■■■■■■2■==========
408             Log.d(TAG, "onCreateView: start loading data, selectedTagNames.size=
409 
410             // ■■■■■■■■■■■onViewCreated■■■RecyclerView■■■■■■■
411 
412             // ■■■■■■■■■■■■■■■■■■■■■■■
413             // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
414             Log.d(TAG, "onCreateView: start async loading tag list");
415             try {
416                 loadTags();
417             } catch (Exception e) {
418                 Log.e(TAG, "onCreateView: failed to load tag list", e);
419                 // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
420             }
421 
422             // ■■■■■■■■■■■■■■■■
423             if (selectedDate != null) {
424                 scrollToDate(recyclerView, selectedDate);
425             }
426 
427             Log.d(TAG, "onCreateView: completed");
428             return root;
429         } catch (Exception e) {
430             Log.e(TAG, "onCreateView: exception occurred", e);
431             throw e;
432         }
433     }
434 
435     /**
436      * ■■■■■■■ + ■■■■ + ■■■■■
437      * ■ debounce ■■■■■■■■■■■■■
438      */
439     private void performSearch(String rawText) {
440         if (adapter == null) return;
441         String text = rawText == null ? "" : rawText.trim();
442         adapter.filter(text);
443         // ■■■■■■■■■■
444         if (!text.isEmpty() && searchHistoryManager != null) {
445             searchHistoryManager.addSearchKeyword(text);
446             refreshHistoryCache();
447             updateInputChrome();
448         }
449     }
450 
```

```
451      /** ■■■■■■■■■■■■■■■■■■■■ */
452      private void refreshHistoryCache() {
453          if (searchHistoryManager == null) return;
454          historyCache = searchHistoryManager.loadHistory();
455          if (historyAdapter != null) {
456              historyAdapter.setItems(historyCache);
457          }
458      }
459
460      /** dp → px ■■■■■ setPadding■ */
461      private int dpToPx(int dp) {
462          return (int) (dp * getResources().getDisplayMetrics().density);
463      }
464
465      /**
466       * ■■■■■■■■■■ × ■■■■■■■■
467       * - ■■■■■■■ ×■■■■■■■
468       * - ■ + ■■ + ■■■■■■■■
469       * - ■ + ■■■■■■■■■■
470       */
471      private void updateInputChrome() {
472          if (searchEditText == null) return;
473          String text = searchEditText.getText().toString();
474          boolean hasText = !text.isEmpty();
475
476          if (searchClearButton != null) {
477              searchClearButton.setVisibility(hasText ? View.VISIBLE : View.GONE);
478          }
479          // ■■ paddingEnd■× ■■ 36dp ■■■× ■■■ 8dp ■■■■
480          // paddingStart / Top / Bottom ■■ 8dp ■■■ XML
481          int endPaddingPx = dpToPx(hasText ? SEARCH_CLEAR_END_PADDING_DP : SEARCH
482          searchEditText.setPaddingRelative(
483              searchEditText.getPaddingStart(),
484              searchEditText.getPaddingTop(),
485              endPaddingPx,
486              searchEditText.getPaddingBottom()
487          );
488
489          if (!hasText && searchEditText.hasFocus() && !historyCache.isEmpty()) {
490              showHistoryPopup();
491          } else {
492              dismissHistoryPopup();
493          }
494      }
495
496      /** ■■■■■■■■ EditText ■■■■■■■■■■■■ */
497      private void showHistoryPopup() {
498          if (searchEditText == null || searchEditText.getWindowToken() == null) r
499          if (historyCache.isEmpty()) return;   // ■■■■■■■■■
500          refreshHistoryCache();               // ■■■■■■■■■■■■■■■■■■■■■
```

```
501          if (historyPopup == null) {
502              buildHistoryPopup();
503          }
504          if (historyPopup != null && !historyPopup.isShowing()) {
505              historyPopup.showAsDropDown(searchEditText, 0, 0);
506          }
507      }
508
509      private void dismissHistoryPopup() {
510          if (historyPopup != null && historyPopup.isShowing()) {
511              historyPopup.dismiss();
512          }
513      }
514
515      /** ■■ PopupWindow■■■■■■■ */
516      private void buildHistoryPopup() {
517          Context ctx = requireContext();
518          historyAdapter = new SearchHistoryAdapter();
519          historyAdapter.setOnItemClickListener(new SearchHistoryAdapter.OnItemCli
520              @Override public void onItemClick(SearchHistoryItem item) {
521                  String kw = item.getText();
522                  searchEditText.removeCallbacks(hidePopupRunnable);
523                  searchEditText.setText(kw);
524                  searchEditText.setSelection(kw.length());
525                  searchEditText.removeCallbacks(debounceSearchRunnable);
526                  dismissHistoryPopup();
527                  // ■■■■■■■■setText ■■■ TextWatcher
528                  performSearch(kw);
529                  updateInputChrome();
530              }
531              @Override public void onPinClick(SearchHistoryItem item) {
532                  searchHistoryManager.togglePinKeyword(item.getText());
533                  refreshHistoryCache();
534                  updateInputChrome();
535              }
536              @Override public void onDeleteClick(SearchHistoryItem item) {
537                  searchHistoryManager.deleteKeyword(item.getText());
538                  refreshHistoryCache();
539                  updateInputChrome();
540              }
541          });
542          historyAdapter.setItems(historyCache);
543
544          RecyclerView rv = new RecyclerView(ctx);
545          rv.setLayoutManager(new LinearLayoutManager(ctx));
546          rv.setAdapter(historyAdapter);
547          rv.setBackgroundResource(R.drawable.search_history_popup_bg);
548          rv.setElevation(8f);
549
550          int width = searchEditText.getWidth();
```

```
551          int maxHeight = (int) (SEARCH_HISTORY_POPUP_MAX_DP * ctx.getResources().
552          historyPopup = new PopupWindow(rv, width, ViewGroup.LayoutParams.WRAP_CO
553          historyPopup.setBackgroundDrawable(new android.graphics.drawable.ColorDr
554          historyPopup.setOutsideTouchable(false);
555          historyPopup.setFocusable(false);  // ■■■■■■■■■■ EditText ■■
556          historyPopup.setHeight(maxHeight);
557
558          // ■■■■■■■■■■■■ pending hide■spec 6.6 ■■■■■
559          rv.setOnTouchListener((v, ev) -> {
560              searchEditText.removeCallbacks(hidePopupRunnable);
561              return false;
562          });
563      }
564
565      @Override
566      public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstance
567          super.onViewCreated(view, savedInstanceState);
568          Log.d(TAG, "onViewCreated: start");
569
570          // ■onViewCreated■■■■■■■■■■■■RecyclerView■■■■■■■■
571          // ■■■■■■■■■■■■■■■■■■■■■■■■■■
572          if (linkDao != null && adapter != null) {
573              Log.d(TAG, "onViewCreated: loading all links directly");
574              List<LinkItem> pinnedLinks = linkDao.getPinnedLinks();
575              Map<String, List<LinkItem>> groupedLinks = linkDao.getLinksGroupByDa
576
577              // ■■■■■■
578              int totalLinks = pinnedLinks.size();
579              for (List<LinkItem> links : groupedLinks.values()) {
580                  totalLinks += links.size();
581              }
582
583              Log.d(TAG, "onViewCreated: loaded links - pinned=" + pinnedLinks.siz
584                      + ", groups=" + groupedLinks.size() + ", total links=" + tot
585
586              // ■■■■■■■adapter
587              adapter.setPinnedLinks(pinnedLinks);
588              adapter.setGroupedLinks(groupedLinks);
589              Log.d(TAG, "onViewCreated: data set to adapter, itemCount=" + adapte
590
591              // ■■RecyclerView■■
592              RecyclerView recyclerView = binding.recyclerView;
593              if (recyclerView != null) {
594                  Log.d(TAG, "onViewCreated: RecyclerView state - visibility=" + r
595                          + " (0=VISIBLE), width=" + recyclerView.getWidth()
596                          + ", height=" + recyclerView.getHeight()
597                          + ", isShown=" + recyclerView.isShown()
598                          + ", hasFixedSize=" + recyclerView.hasFixedSize());
599
600                  // ■■RecyclerView■■
```

```
601                 if (recyclerView.getVisibility() != View.VISIBLE) {
602                     Log.w(TAG, "onViewCreated: RecyclerView is not visible, sett
603                     recyclerView.setVisibility(View.VISIBLE);
604                 }
605 
606                 // ■■LayoutManager
607                 RecyclerView.LayoutManager layoutManager = recyclerView.getLayou
608                 if (layoutManager != null) {
609                     Log.d(TAG, "onViewCreated: LayoutManager exists, itemCount="
610                 } else {
611                     Log.e(TAG, "onViewCreated: LayoutManager is null!");
612                 }
613 
614                 // ■■■■
615                 recyclerView.post(() -> {
616                     Log.d(TAG, "onViewCreated: post - RecyclerView width=" + rec
617                             + ", height=" + recyclerView.getHeight()
618                             + ", adapter itemCount=" + (adapter != null ? adapte
619                     if (adapter != null) {
620                         adapter.notifyDataSetChanged();
621                         Log.d(TAG, "onViewCreated: post - notifyDataSetChanged c
622                     }
623                 });
624             } else {
625                 Log.e(TAG, "onViewCreated: RecyclerView is null!");
626             }
627         } else {
628             Log.e(TAG, "onViewCreated: linkDao or adapter is null!");
629         }
630 
631         // ■■■■■■■■■■■■■■■■■■■
632         if (getArguments() != null) {
633             String selectedDate = getArguments().getString("selected_date");
634             if (selectedDate != null && binding != null) {
635                 RecyclerView recyclerView = binding.recyclerView;
636                 if (recyclerView != null) {
637                     scrollToDate(recyclerView, selectedDate);
638                 }
639             }
640         }
641 
642         Log.d(TAG, "onViewCreated: completed");
643     }
644 
645     // ========== ■■■■■■■■■■■ ==========
646 
647     /**
648      * ■■■■■RecyclerView
649      * ■TagsFragment■■■■■2■■■■■■■■
650      */
```

```
651      private void initTagRecyclerView(View root) {
652          Log.d(TAG, "initTagRecyclerView: start");
653          try {
654              if (root == null) {
655                  Log.e(TAG, "initTagRecyclerView: root is null!");
656                  return;
657              }
658
659              // ■■■■■■■■■■2■■■■■■
660              tagsContainer = root.findViewById(R.id.tags_container);
661              if (tagsContainer != null) {
662                  tagsContainer.setVisibility(View.VISIBLE);  // ■■2■■■■■■■■■■
663                  Log.d(TAG, "initTagRecyclerView: tags container displayed");
664              } else {
665                  Log.w(TAG, "initTagRecyclerView: tagsContainer is null, may not
666              }
667
668              // ■■■■■■■■ RecyclerView
669              Log.d(TAG, "initTagRecyclerView: initializing fixed tags RecyclerVie
670              tagsRecyclerView = root.findViewById(R.id.recycler_tags_fixed);
671              if (tagsRecyclerView != null) {
672                  try {
673                      FlexboxLayoutManager layoutManager = new FlexboxLayoutManage
674                      layoutManager.setFlexDirection(FlexDirection.ROW);
675                      layoutManager.setFlexWrap(FlexWrap.WRAP);
676                      tagsRecyclerView.setLayoutManager(layoutManager);
677
678                      tagsAdapter = new TagsAdapter();
679                      tagsAdapter.setOnTagClickListener((position, tag) -> {
680                          if (!isSortMode) {
681                              handleTagClick(tag.getName());
682                          }
683                      });
684                      tagsAdapter.setOnTagLongClickListener((position, tag) -> {
685                          if (!isSortMode) {
686                              showTagOptionsDialog(tag.getName());
687                          }
688                      });
689                      tagsRecyclerView.setAdapter(tagsAdapter);
690                      Log.d(TAG, "initTagRecyclerView: fixed tags RecyclerView ini
691                  } catch (Exception e) {
692                      Log.e(TAG, "initTagRecyclerView: fixed tags RecyclerView ini
693                  }
694              } else {
695                  Log.w(TAG, "initTagRecyclerView: recycler_tags_fixed is null");
696              }
697
698              // ■■■■■■■■ RecyclerView
699              Log.d(TAG, "initTagRecyclerView: initializing collapsed tags Recycle
700              tagsRecyclerViewCollapsed = root.findViewById(R.id.recycler_tags_col
```

```
701             if (tagsRecyclerViewCollapsed != null) {
702                 try {
703                     FlexboxLayoutManager layoutManagerCollapsed = new FlexboxLay
704                     layoutManagerCollapsed.setFlexDirection(FlexDirection.ROW);
705                     layoutManagerCollapsed.setFlexWrap(FlexWrap.WRAP);
706                     tagsRecyclerViewCollapsed.setLayoutManager(layoutManagerColl
707 
708                     tagsAdapterCollapsed = new TagsAdapter();
709                     tagsAdapterCollapsed.setOnTagClickListener((position, tag) -
710                         if (!isSortMode) {
711                             handleTagClick(tag.getName());
712                         }
713                     });
714                     tagsAdapterCollapsed.setOnTagLongClickListener((position, ta
715                         if (!isSortMode) {
716                             showTagOptionsDialog(tag.getName());
717                         }
718                     });
719                     tagsRecyclerViewCollapsed.setAdapter(tagsAdapterCollapsed);
720                     Log.d(TAG, "initTagRecyclerView: collapsed tags RecyclerView
721                 } catch (Exception e) {
722                     Log.e(TAG, "initTagRecyclerView: collapsed tags RecyclerView
723                 }
724             } else {
725                 Log.w(TAG, "initTagRecyclerView: recycler_tags_collapsed is null
726             }
727 
728         // ■■■■■■■■■■■■
729         expandMoreTagsButton = root.findViewById(R.id.btn_expand_more_tags);
730         if (expandMoreTagsButton != null) {
731             textExpandMore = root.findViewById(R.id.text_expand_more);
732             iconExpandMore = root.findViewById(R.id.icon_expand_more);
733             expandMoreTagsButton.setOnClickListener(v -> toggleMoreTagsExpansion
734         }
735 
736         // ■■■ ItemTouchHelper■■■■■■■■■
737         if (tagsRecyclerView != null) {
738             ItemTouchHelper.Callback callback = new ItemTouchHelper.SimpleCallba
739                 ItemTouchHelper.UP | ItemTouchHelper.DOWN |
740                 ItemTouchHelper.LEFT | ItemTouchHelper.RIGHT,
741                 0
742             ) {
743                 @Override
744                 public boolean onMove(@NonNull RecyclerView recyclerView,
745                                       @NonNull RecyclerView.ViewHolder viewHolder
746                                       @NonNull RecyclerView.ViewHolder target) {
747                     if (isSortMode && recyclerView == tagsRecyclerView) {
748                         int fromPos = viewHolder.getAdapterPosition();
749                         int toPos = target.getAdapterPosition();
750                         tagsAdapter.swapItems(fromPos, toPos);
```

```
751                         saveTagOrderToDatabase();
752                         return true;
753                     }
754                     return false;
755                 }
756 
757                 @Override
758                 public void onSwiped(@NonNull RecyclerView.ViewHolder viewHolder
759                     // ■■■■■■
760                 }
761 
762                 @Override
763                 public boolean isLongPressDragEnabled() {
764                     return isSortMode;  // ■■■■■■■■■■
765                 }
766             };
767             itemTouchHelper = new ItemTouchHelper(callback);
768             itemTouchHelper.attachToRecyclerView(tagsRecyclerView);
769         }
770 
771         // ■■■■■■■■■■/■■■■■■■
772         tagsScrollView = root.findViewById(R.id.tags_scrollview);
773         toggleTagsButton = root.findViewById(R.id.btn_toggle_tags);
```